



COURS: LANGAGE C (PARTIE_1)

Par: Dr. Mohamed Lemine Sidi



CONTENU DU COURS (PARTIE_1)

1. Introduction
2. Les types des variables
3. Les opérateurs
4. Les instructions conditionnelles





RESSOURCES ET OUTILS

Livres:

- Canteaut, Anne. "Programmation en langage C." *INRIA-projet CODES* (2003).
- Aitken, Peter, and Bradley Jones. *Le langage C*. Pearson Education France, 2008.
- Kernighan, Brian W., and Dennis M. Ritchie. *The C programming language*. prentice-Hall, 1988.

Outils (Pour les TPs):

- Dev-C++





1. INTRODUCTION

Le langage C est un langage de programmation puissant et polyvalent, utilisé dans de nombreux domaines, le développement de systèmes d'exploitation, la programmation embarquée, le développement de jeux video, le développement des pilotes de périphériques, etc.

Historique:

- Création : Développé par Dennis Ritchie dans les laboratoires Bell Labs au début des années 1970.

Les Caractéristiques clés :

- Langage de bas niveau : Permet un contrôle précis de la mémoire et des ressources matérielles.
- Portable : Les programmes C peuvent être compilés et exécutés sur différentes plateformes.
- Efficace : Connu pour sa rapidité d'exécution.
- Structuré : Encourage la programmation modulaire et organisée.



1. INTRODUCTION -> COMPILEATION

1. La compilation

- Un programme C doit être compilé avant d'être exécuté. Le compilateur C transforme le programme écrit en C en un fichier exécutable que l'ordinateur peut exécuter directement.
- Le rôle principal du compilateur C est de traduire le **code source** écrit en langage C (compréhensible par l'homme) en **code machine** ou **code binaire** (compréhensible par l'ordinateur). Ce processus est essentiel car les ordinateurs ne peuvent exécuter que des instructions en langage machine.

Code source

```
#include<stdio.h>
main(){
    printf("Bonjour");
    .
    .
    .
}
```

Le compilateur C

Code machine

```
100101001110111000001101
010010001101111111111111
.
.
.
.
101010110010111111110110
```



1. INTRODUCTION -> LA STRUCTURE D'UN PROGRAMME C

2. La structure d'un programme C:

```
#include<stdio.h>    // Inclut la bibliothèque standard d'entrée/sortie
int main(){
    printf("Bonjour tout le monde");
    return 0;        // Indique que le programme s'est terminé avec succès.
}
```

Elle se compose de plusieurs parties.

- **Les inclusions de bibliothèques (#include) :**
 - Cette directive (#include) permet d'inclure des fichiers (bibliothèques) qui contiennent des des fonctions pré-écrites nécessaires vos programmes.
 - Par exemple, **#include <stdio.h>** inclut la bibliothèque standard d'entrée/sortie, qui fournit des fonctions comme **printf()** et **scanf()**.
- **La fonction main :** C'est le point d'entrée du programme.
 - Tout programme C commence son exécution par la fonction main.
 - Elle peut retourner un entier (int) pour indiquer le statut de fin du programme (0 pour succès).



1. INTRODUCTION -> LA STRUCTURE D'UN PROGRAMME C

- **Déclarations de variables:**

- Les variables doivent être déclarées avant d'être utilisées.
- Elles stockent des données temporaires pendant l'exécution du programme.
- Exemple: C utilise différents types de données pour stocker des informations, comme **int** pour les entiers, **float** pour les nombres à virgule flottante, **char** pour les caractères, etc.

- **Les Instructions :**

- Les instructions sont des actions que le programme exécute, comme des calculs, des appels de fonctions, etc.
- Par exemple: `printf("Bonjour tout le monde");`, c'est une instruction qui permet d'afficher la phrase « Bonjour tout le monde » sur l'écran de l'ordinateur.

NB: Les instructions en C se terminent par un point-virgule (;).

- Les blocs de code sont délimités (début et fin) par des accolades ({ }).
- Les commentaires peuvent être ajoutés en utilisant `//` (commentaire sur une seule ligne) ou `/* ... */` (commentaire sur plusieurs lignes).



2. LES TYPES DES VARIABLES

En langage C, les variables sont typées, ce qui signifie que chaque variable doit avoir un type de données spécifique qui détermine la nature des valeurs qu'elle peut stocker. Voici les types de base des variables en C :

2.1. Types des entiers:

- Utilisés pour stocker des nombres entiers (positifs ou négatifs).
- La taille et la plage de valeurs dépendent du type et de l'architecture du système.

Type	Description	Taille (en bits)	Plage de valeurs
int	Entier standard.	16 ou 32	-32 768 à 32 767 ou -2 147 483 648 à 2 147 483 647
short	Entier court.	16	-32 768 à 32 767
long	Entier long (généralement 4 ou 8 octets selon l'architecture).	32 ou 64	-2 147 483 648 à 2 147 483 647 ou plus
long long	Entier très long.	64	Très grande plage (environ -9×10^{18} à 9×10^{18})



2. LES TYPES DES VARIABLES

2.2. Types flottants:

- Utilisés pour stocker des nombres réels (avec une partie décimale).
- Ils sont représentés en virgule flottante.

Type	Description	Taille (en bits)	Plage de valeurs
float	Nombre à virgule flottante, précision simple.	32	1.2E-38 à 3.4E+38 (approximativement)
double	Nombre à virgule flottante, double précision.	64	2.3E-308 à 1.7E+308 (approximativement)
long double	Entier long (généralement 4 ou 8 octets selon l'architecture).	80 ou 128	Dépend de l'implémentation du compilateur (généralement jusqu'à 1.1E+4932 à 1.1E+4932 pour la version 80 bits)

2. LES TYPES DES VARIABLES

2.3. Types caractère:

- Utilisé pour stocker un seul caractère (lettre, chiffre, symbole).
- Le type char est en réalité un entier de 8 bits qui représente un caractère selon la table ASCII.

Type	Description	Taille (en bits)	Plage de valeurs
char	Caractère ASCII ou petit entier.	8	-128 à 127 (signé), 0 à 255 (non signé)

NB: La table ASCII (American Standard Code for Information Interchange) est une norme qui associe chaque caractère à un nombre entier. Par exemple :

- Le caractère 'A' est associé à l'entier 65.
- Le caractère 'b' est associé à l'entier 98.
- Le caractère '1' est associé à l'entier 49.

Exemple :

```
char c = 'A';  
printf("%d", c); // Affichera 65  
printf("%c", c); // Affichera A
```



2. LES TYPES DES VARIABLES

2.4. Modificateurs de types:

- Les types modificateurs peuvent être utilisés pour modifier la plage des types de données de base. Ils permettent de préciser si la variable peut être positive et négative ou uniquement positive, ou bien de définir des tailles plus grandes ou plus petites pour les types entiers.
- Les principaux modificateurs sont :
 - **signed** : Indique que la variable peut contenir des valeurs négatives et positives (par défaut pour int).
 - **unsigned** : Indique que la variable ne peut contenir que des valeurs positives (ou zéro), ce qui augmente la plage des valeurs possibles.

Exemple:

```
signed int a = -10;    // Entier signé
unsigned int b = 20;   // Entier non signé
```

- **short** : Réduit la taille du type.
- **long** : Augmente la taille du type.

Exemple:

```
long int a = 100000000; // Entier long
short int b = 32767;     // Entier court
```



3. LES OPÉRATEURS

En langage C, les opérateurs sont des symboles utilisés pour effectuer des opérations sur des variables et des valeurs. Ils jouent un rôle essentiel dans la manipulation des données.

3.1. Opérateurs arithmétiques

- Utilisés pour effectuer des calculs mathématiques de base.

Opérateur	Description	Exemple
+	Addition	$a + b$
-	Soustraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b

3. LES OPÉRATEURS

3.2. Opérateurs de comparaison

- Utilisés pour comparer deux valeurs. Le résultat est un booléen (1 pour vrai, 0 pour faux).

Opérateur	Description	Exemple
==	Égal à	$a == b$
!=	Différent de	$a != b$
>	Supérieur à	$a > b$
<	Inférieur à	$a < b$
>=	Supérieur ou égal à	$a \geq b$



3. LES OPÉRATEURS

3.3. Opérateurs logiques

- Utilisés pour combiner des conditions.

Opérateur	Description	Exemple
&&	ET logique (AND)	$a > b \ \&\& \ c < d$ (vrai si $a > b$ et $c < d$)
	OU logique (OR)	$a \ \ b$ (vrai si a ou b est vrai)
!	Négation (NOT)	$!(a > b)$ (vrai si $a \leq b$)

3. LES OPÉRATEURS

3.4. Opérateurs d'affectation

- Utilisés pour affecter une valeur à une variable.

Opérateur	Description	Exemple
=	Affectation simple	$a = b$
+=	Ajout et affectation	$a += b$ ($a = a + b$)
-=	Soustraction et affectation	$a -= b$ ($a = a - b$)
*=	Multiplication et affectation	$a *= b$ ($a = a * b$)
/=	Division et affectation	$a /= b$ ($a = a / b$)
%=	Modulo et affectation	$a \% = b$ ($a = a \% b$)



3. LES OPÉRATEURS

3.5. Opérateurs d'incrémentation et de décrémentation

- Utilisés pour augmenter ou diminuer une variable de 1.

Opérateur	Description	Exemple
++	Incrémentation	a++ ou ++a
--	Décrémentation	a-- ou --a

NB:

- Pré-incrémentation (++a) : La valeur est incrémentée avant d'être utilisée.
- Post-incrémentation (a++) : La valeur est utilisée avant d'être incrémentée.

4. INSTRUCTIONS CONDITIONNELLES

Les instructions conditionnelles sont des structures de contrôle qui permettent d'exécuter certaines parties du code en fonction de conditions spécifiques. Elles évaluent une expression booléenne (vrai ou faux) et exécutent des blocs de code en fonction du résultat de cette évaluation.

4.1. L'instruction « **if - else** »

- L'instruction **if** évalue une expression **booléenne**. Si l'expression est vraie (non nulle), le bloc de code associé à l'instruction **if** est exécuté.
- L'instruction **else** est utilisée en conjonction avec l'instruction **if**. Si l'expression booléenne de l'instruction **if** est fausse (nulle), le bloc de code associé à l'instruction **else** est exécuté.

Syntaxe :

```
if (expression){  
    // Bloc de code à exécuter si l'expression est vraie  
}else{  
    // Bloc de code à exécuter si l'expression est fausse  
}
```



4. INSTRUCTIONS CONDITIONNELLES

4.2. L'instruction « else if »

- L'instruction **else if** permet de tester plusieurs conditions en séquence. Elle est utilisée après une instruction **if** ou **else if** précédente.

Syntaxe :

```
if (expression1){  
    // Bloc de code à exécuter si expression1 est vraie  
}else if (expression2){  
    // Bloc de code à exécuter si expression2 est vraie  
}else{  
    // Bloc de code à exécuter si toutes les expressions précédentes sont fausses  
}
```



4. INSTRUCTIONS CONDITIONNELLES

Exemple : Un programme qui demande a l'utilisateur d'entrer une note et lui retourne la mention.

```
#include<stdio.h>
main(){
    int note;
    printf("Donnez une note[0 20]:\n"); scanf("%d",&note);
    if (note<0 || note>20){
        printf("la note doit etre entre 0 et 20");
    }else{
        if (note >= 16) {
            printf("Tres bien \n");
        } else if (note >= 14) {
            printf("Bien \n");
        } else if (note >= 10) {
            printf("Passable.\n");
        } else {
            printf("Ajourne.\n");
        }
    }
}
```



4. INSTRUCTIONS CONDITIONNELLES

4.3. L'instruction « **switch** »

- L'instruction **switch** permet de tester une variable par rapport à plusieurs valeurs possibles. Elle est plus efficace que l'utilisation de plusieurs instructions if imbriquées lorsque vous devez tester une variable par rapport à un grand nombre de valeurs.

Syntaxe :

```
switch (variable) {  
  case valeur1:  
    // Bloc de code à exécuter si variable == valeur1  
    break;  
  case valeur2:  
    // Bloc de code à exécuter si variable == valeur2  
    break;  
  default:  
    // Bloc à exécuter si variable ne correspond à aucune des valeurs précédentes  
}
```



4. INSTRUCTIONS CONDITIONNELLES

NB:

- L'instruction **break** est essentielle dans une instruction **switch**. Elle permet de sortir du bloc switch après l'exécution du bloc de code correspondant à la valeur de la variable.
- Sans l'instruction **break**, l'exécution continuerait dans les blocs de code suivants.

Exercice: Ecrivez un programme « calculatrice » qui effectue les opérations élémentaires (+, -, *, /), en utilisant l'instruction conditionnelle switch.